sfxPackager v4.0

User’s Guide

This guide is intended to show you how to build install packages for Windows using sfxPackager.

The latest release and source code is available at either:
https://sourceforge.net/projects/sfxpackager/
...or...
https://github.com/keelanstuart/sfxPackager

Intro

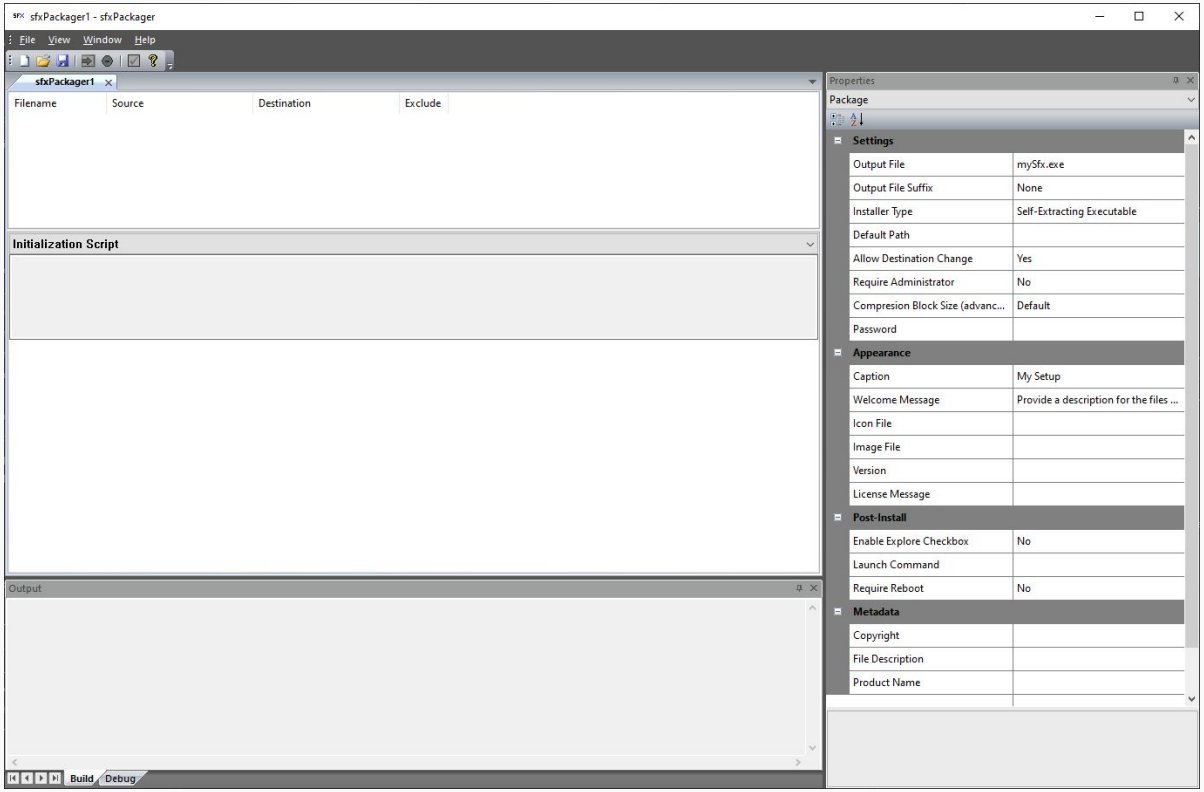
sfxPackager was designed to help you create installers for Windows platforms quickly and easily. It integrates a simple JavaScript interpreter as well as other features to customize the look and behavior of your setup.

Follow along with this tutorial to learn how to accomplish common (and some uncommon) tasks.

Project-based development

sfxPackager saves and loads files with the extension .sfxpp - short for “sfxPackager Project”.

The idea is that you build your project once and then when you need to update your installer, you load your sfxpp, click the build button, and voila! A new setup exe is generated for you.

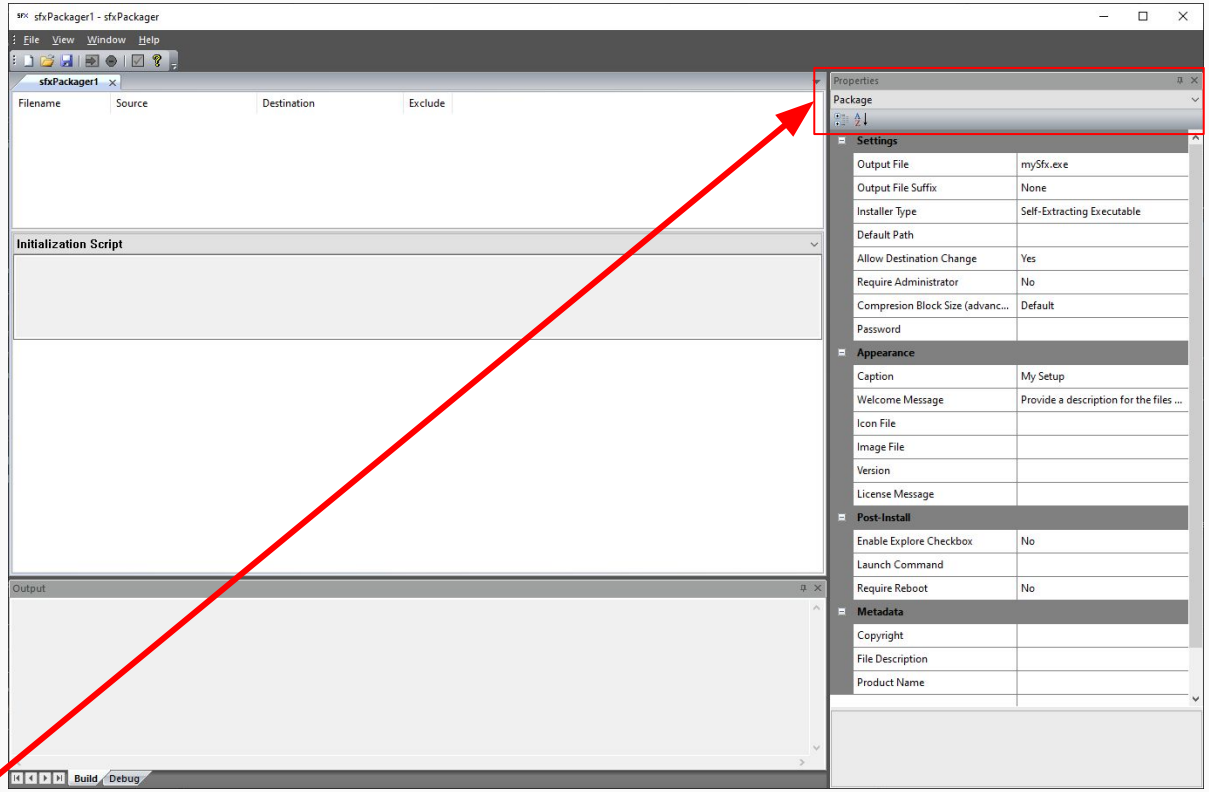


Setting Types

Settings for your project are divided into three parts:

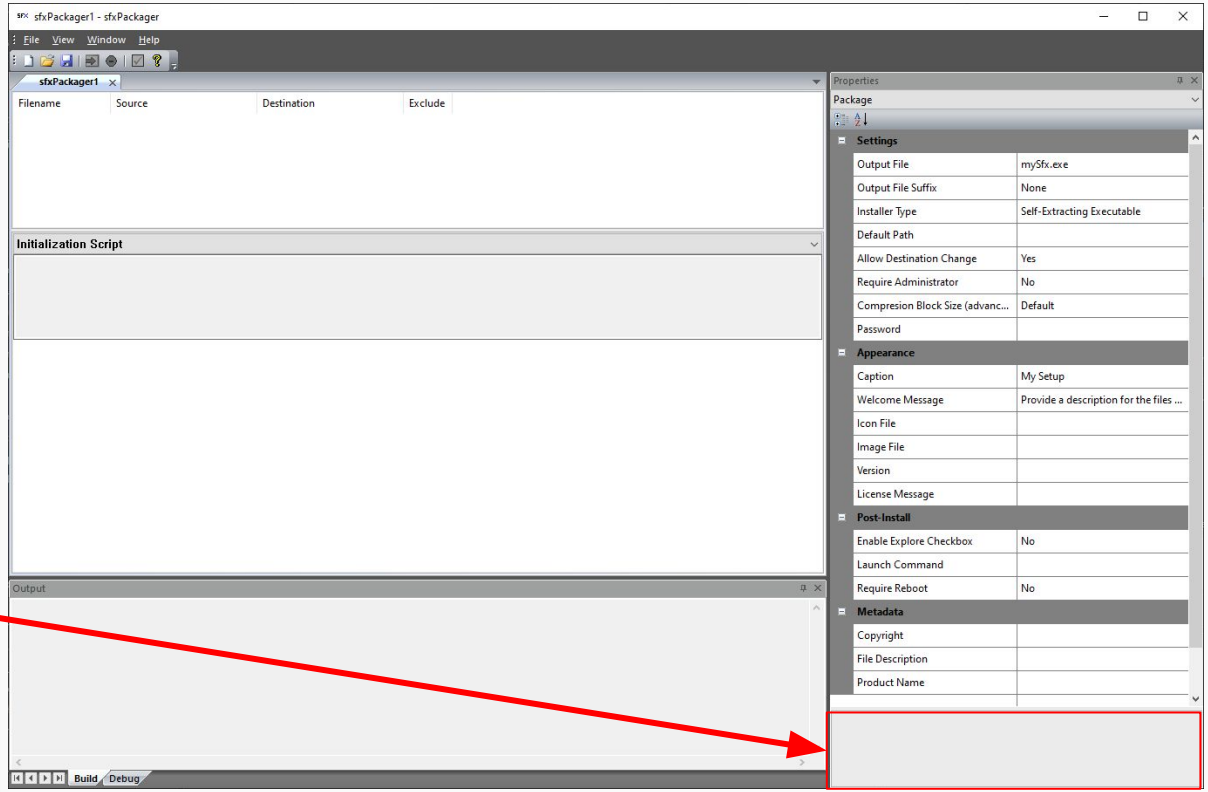
- **Package** settings that are applied to your installer as a whole, such as the output filename, window icon, image, etc.
- **File** settings for each entry in your project, such as the source and destination
- **Settings** settings, which are for the packaging application itself

You can change which setting type you are viewing with the dropdown, here...



Online Help

You can always see what any particular setting does by looking at the help text displayed at the bottom of the properties window



Package Settings

Settings Group

- **Output File:** Specifies the output executable file that will be created when this project is built
- **Default Path:** The default root path where the install data will go
- **Allow Destination Change:** Allow the user to change the install destination folder
- **Require Administrator:** If set, requires the user to have administrative privileges and will prompt the user to elevate if they do not
- **Output File Suffix:** Allows suffix to be automatically added immediately before the output file names extension. You can choose no suffix, the version (maj.min.rel.bld format) or the current date (YYYYMMDD format)
- **Installer Type:** Determines whether the package data is stored inside the exe or in another data file; if your archive exceeds 4GB, use an external data file
- **Compression Block Size:** Specifies the size of pre-compression blocks, ranging from 8KB to 256KB. Some data may compress better in larger or smaller blocks. WARNING: ADJUST AT YOUR OWN RISK. Empirically, the *smallest* block size seems to result in the smaller overall output file
- **Password**: Optionally supply a password. Your archive will be encrypted using SHA-256, inaccessible without the password.

Package Settings

Appearance Group

- **Caption:** Specifies the text that will be displayed in the window's title bar
- **Version:** Specifies the version number that will be displayed by the installer. This is EITHER a string literal or the path to an .EXE, from which a version number will be extracted
- **Welcome Message:** Specifies the text that will be displayed in the window's main area to tell the end-user what the package is. May contain HTML in-line or reference a filename that contains HTML content
- **License Message:** OPTIONAL: Specifies the text that will be displayed on the license dialog when the Javascript GetLicenseKey function is called. May contain HTML in-line or reference a filename that contains HTML content. If the JS function is never called, this goes unused
- **Icon File:** Specifies the ICO-format window icon
- **Image File:** Specifies a BMP-format image that will be displayed on the window (the default size is 200 x 400)

Package Settings

Post-Install Group

- **Enable Explore Checkbox:** If set, will display a checkbox on the completion dialog that will open a windows explorer window to the install directory upon completion if checked
- **Launch Command:** If set, will display a checkbox, optionally allowing a command to be executed when installation is complete (see full docs for parameter options). This is more or less deprecated, since the same effect can be achieved with a post-install script
- **Require Reboot:** If set, will inform that user that a reboot of the system is recommended and prompt to do that

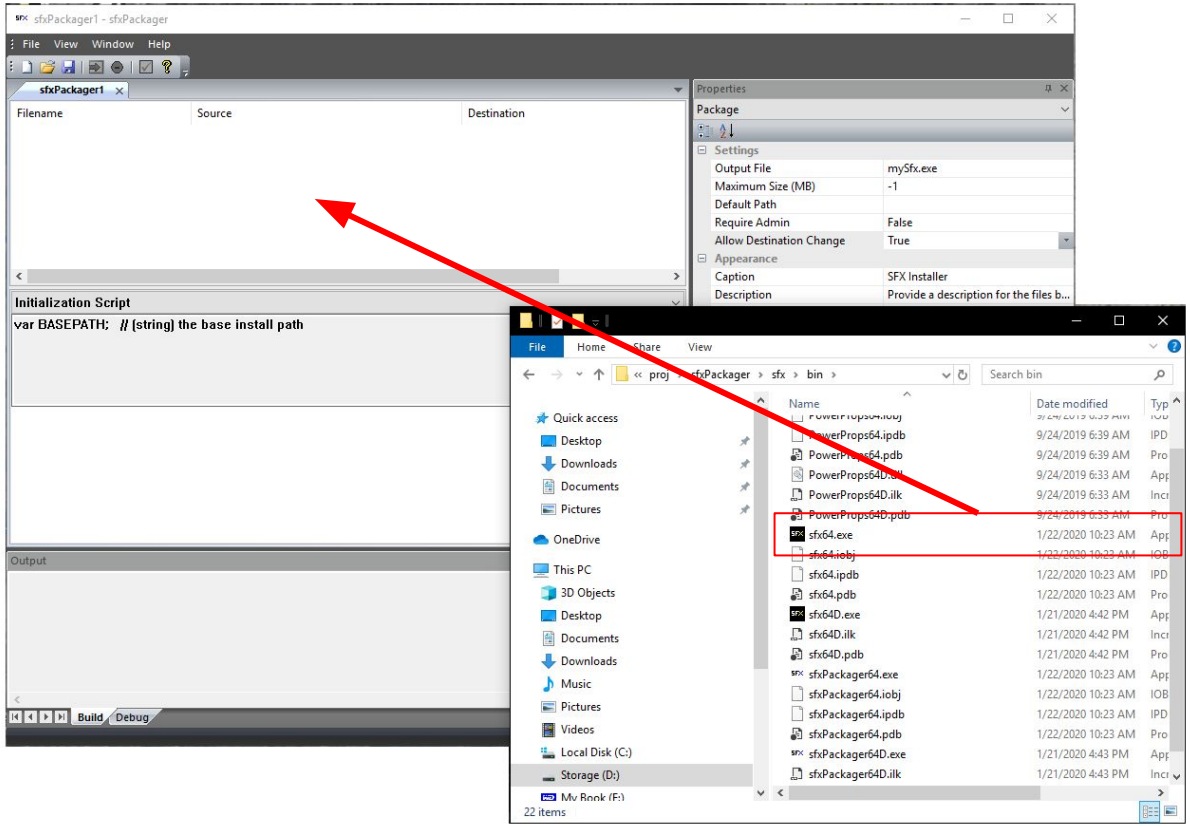
Package Settings

Metadata Group

- **Copyright:** Sets the Copyright field in the installer executable version info, as visible from the Properties->Details tab in Windows Explorer
- **File Description:** Sets the File Description field in the installer executable version info, as visible from the Properties->Details tab in Windows Explorer
- **Product Name:** Sets the Product Name field in the installer executable version info, as visible from the Properties->Details tab in Windows Explorer

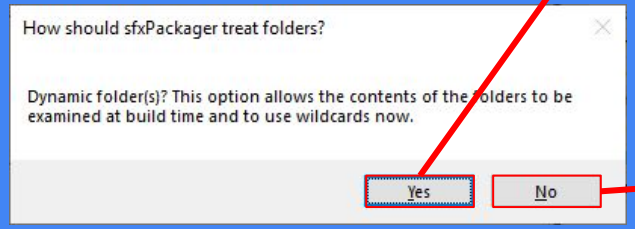
Adding Files

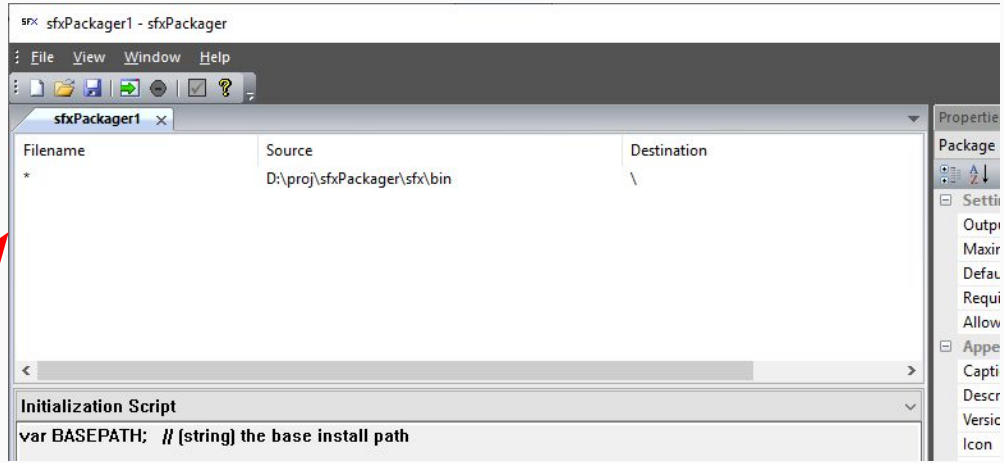
Adding files can be done in one of two ways: either by dragging files or directories to the file panel or by right-clicking the file panel and selecting “Add New File”.

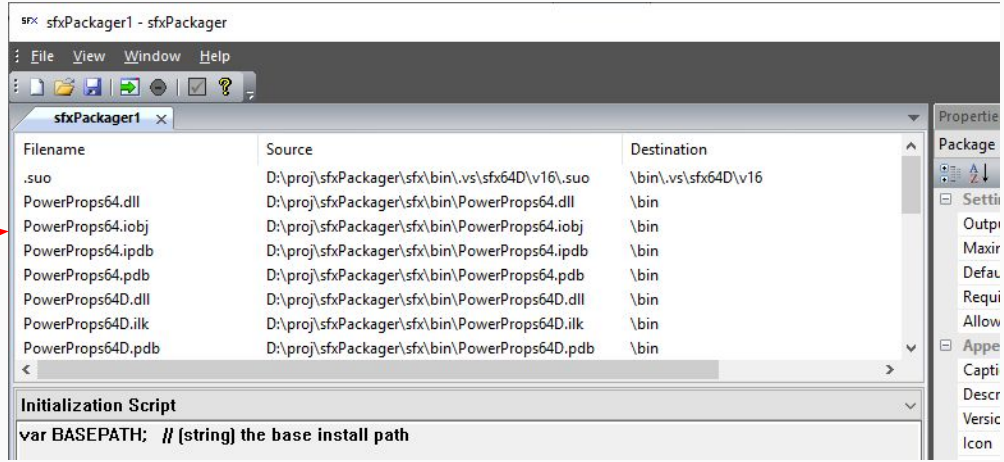


Adding Directories

If you add a directory to your project, you will be prompted to either add all files now or to evaluate them when the project is built. This can help eliminate sfxpp changes when your new build ships different files.





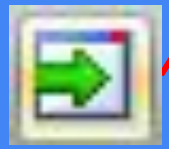


File Settings

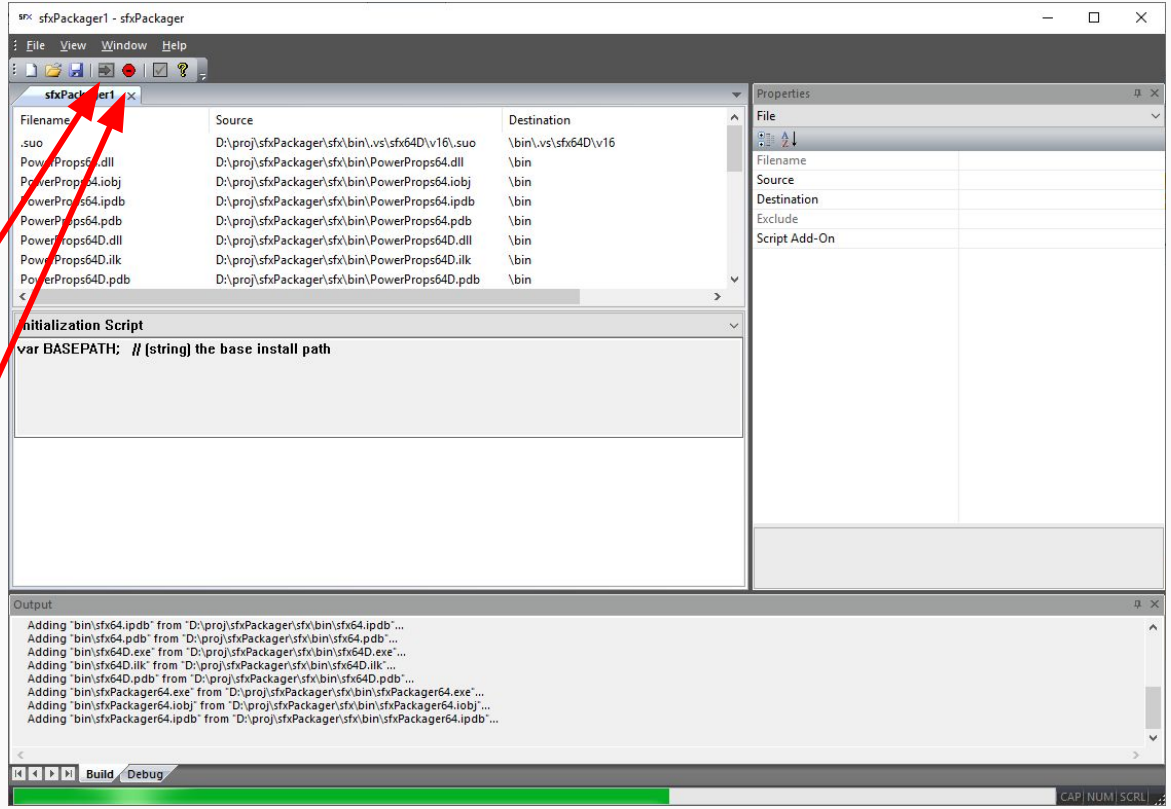
- **Filename**: The name of the file that will be installed. Some notes:
  - May contain wildcards
  - The filename can, if it is specific (i.e., does not contain wildcards), be different than the source filename, effectively renaming it when installed
- **Source**: The source location of the file(s) to be installed; can be:
  - a directory with a file specification (wildcards ok)
  - a single file
  - a URL in “http://somewhere.com/somefile”
- **Destination**: The absolute or relative path (from the directory the user chooses) where the file(s) will be installed.
- **Exclude**: If you provide wildcards, you can exclude files with this semi-colon delimited list of file specifications (wildcards, too)
- **Pre-Install Script Snippet**: Covered later in more detail, but you can provide a JavaScript snippet that will be appended to the Pre-Install, Per-File script and run before this file, or group of files, is decompressed. In this script, you can call the “Skip()” function to ignore the file without decompressing it
- **Post-Install Script Snippet**: Covered later in more detail, but you can provide a JavaScript snippet that will be appended to the Post-Install, Per-File script and run after this file, or group of files, is decompressed.

Building Your Project

Once you have added all the files required for your installation, click the Build button to begin the build process, after which you should see an .exe at the location provided in your Package Settings



You may stop an active build at any time by clicking the Cancel button

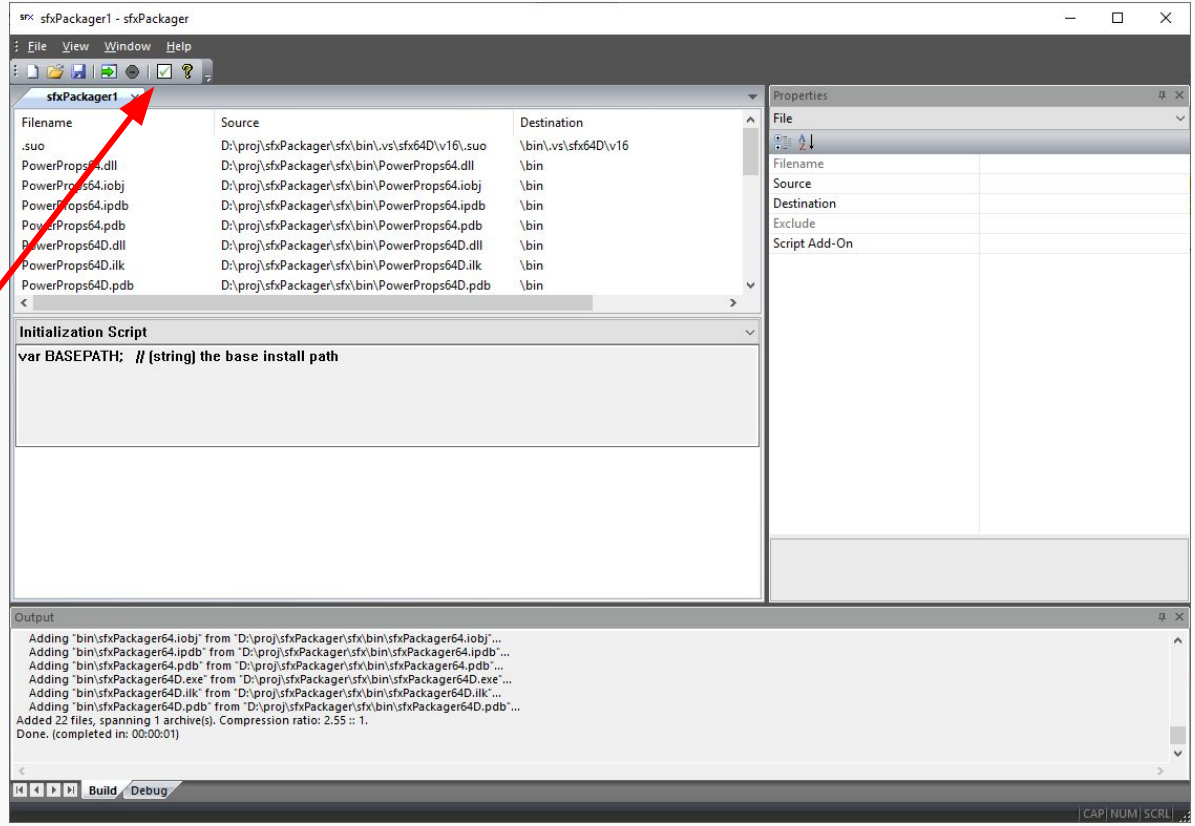


Testing Your Project

After a build has been successfully completed, you may test it by clicking the Test buton.

The Test button will only appear once the exe file exists.

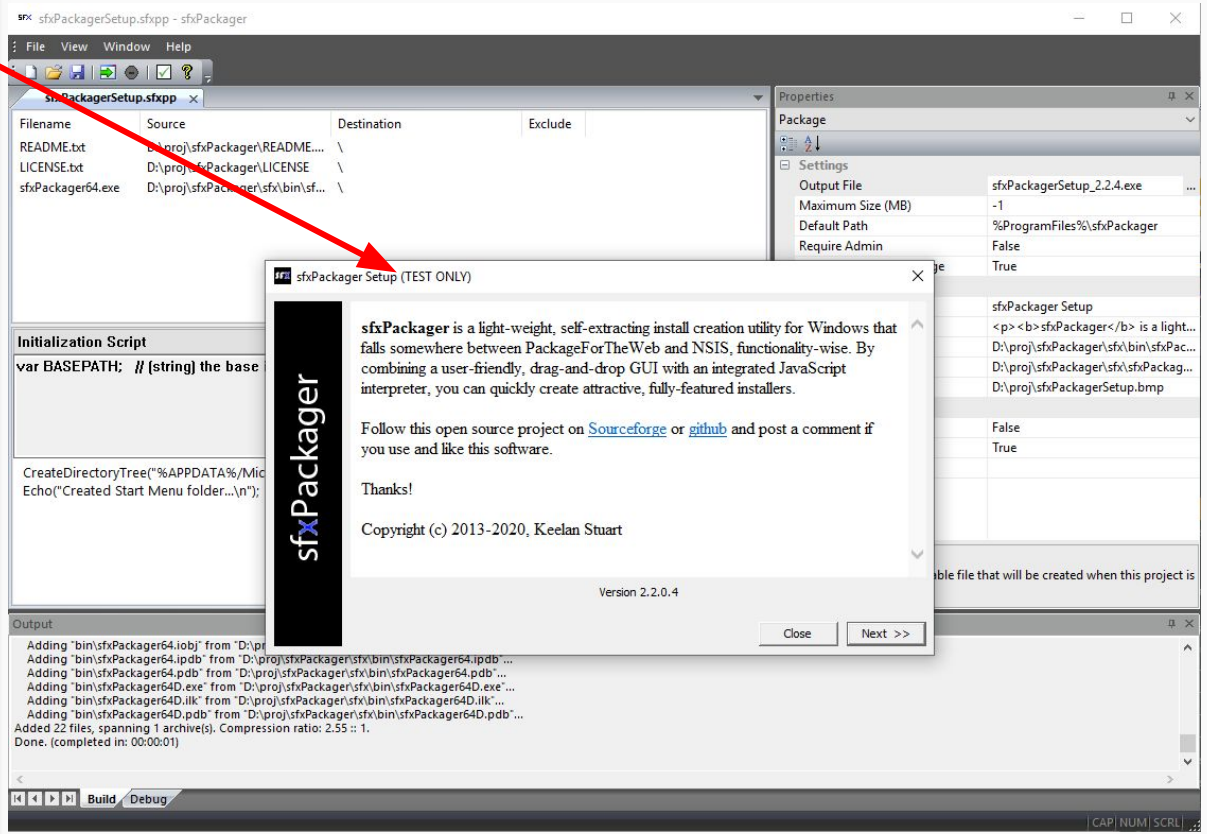
Your installer will be run with the “-testonly” command line parameter, which will only simulate the actions the installer would take if run normally.



(TEST ONLY)

If you see this text in the caption of your installer program, you know that no actual file operations will take place.

Test only mode is useful for making sure your files will go where you expect and to see that your scripts are functioning as expected.



Important Note

When supplying almost ANY filename, you can embed either **Environment Variables** or **Registry Values** by bracketing your text in '%' or '@' characters, respectively.

So, to set the default install location for your files to directory under *C:\Program Files*, say, *MyProgram* you can use the Environment Variable "ProgramFiles" in your path like this: "%ProgramFiles%\MyProgram". To see what variables are available, hit Win+R and run "cmd" to open a command prompt, then type "set" and press enter.

To embed a system Registry Value a similar procedure is followed. Give a registry path like "@HKEY_CURRENT_USER\somePath\SomeKey@". To view registry keys, hit Win+R and run "regedit".

A special, sfxPackager-only environment value is "CWD" and refers to the directory where the package exe was run from. As with other environment variables, use the "%cwd%" form.

Automation

sfxPackager can be run from the command line to automate package creation operations. For example, your project could have a post-build step that launches sfxPackager to make your distributable installer.

To accomplish this, use the “/b <sfxpp file>” option on the command line when running sfxPackager. If you require blocking, try something like:

```
> start /WAIT sfxPackager64 /b d:\proj\sfxPackagerSetup.sfxpp
```

Scripting *(NOTE: this guide is not intended to teach JavaScript programming!)*

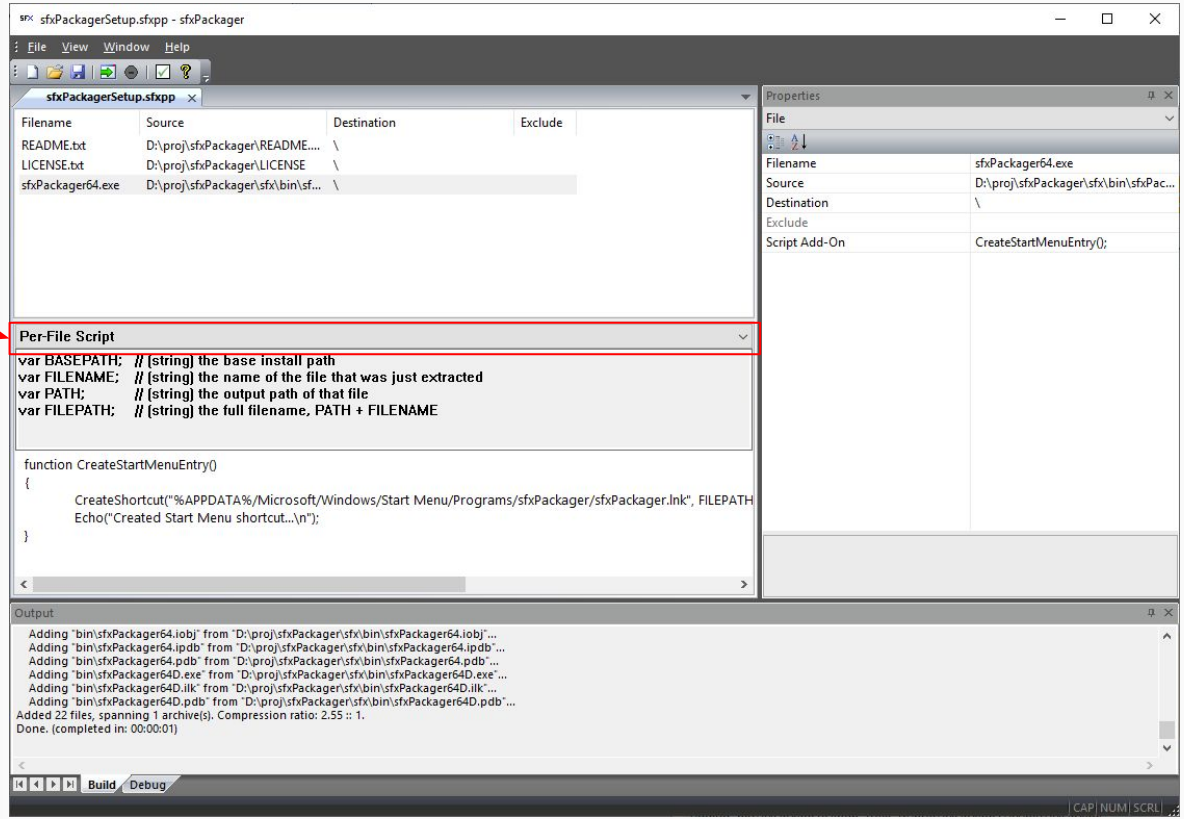
sfxPackager includes a JavaScript interpreter and some embedded functions that allow you to extend the capabilities of your installer. There are five events that cause JS code to be run:

- **Initialization**: *Prior to any dialogs being shown; primarily for setting up user-editable properties in the Configuration dialog*
- **Pre-Install**: *After the Configuration dialog's "Next" button has been clicked, but before any files have been installed*
- **Pre-File**: *Before an individual file is installed*
- **Post-File**: *After an individual file has been installed*
- **Post-Install**: *After all files have been installed*

Scripting

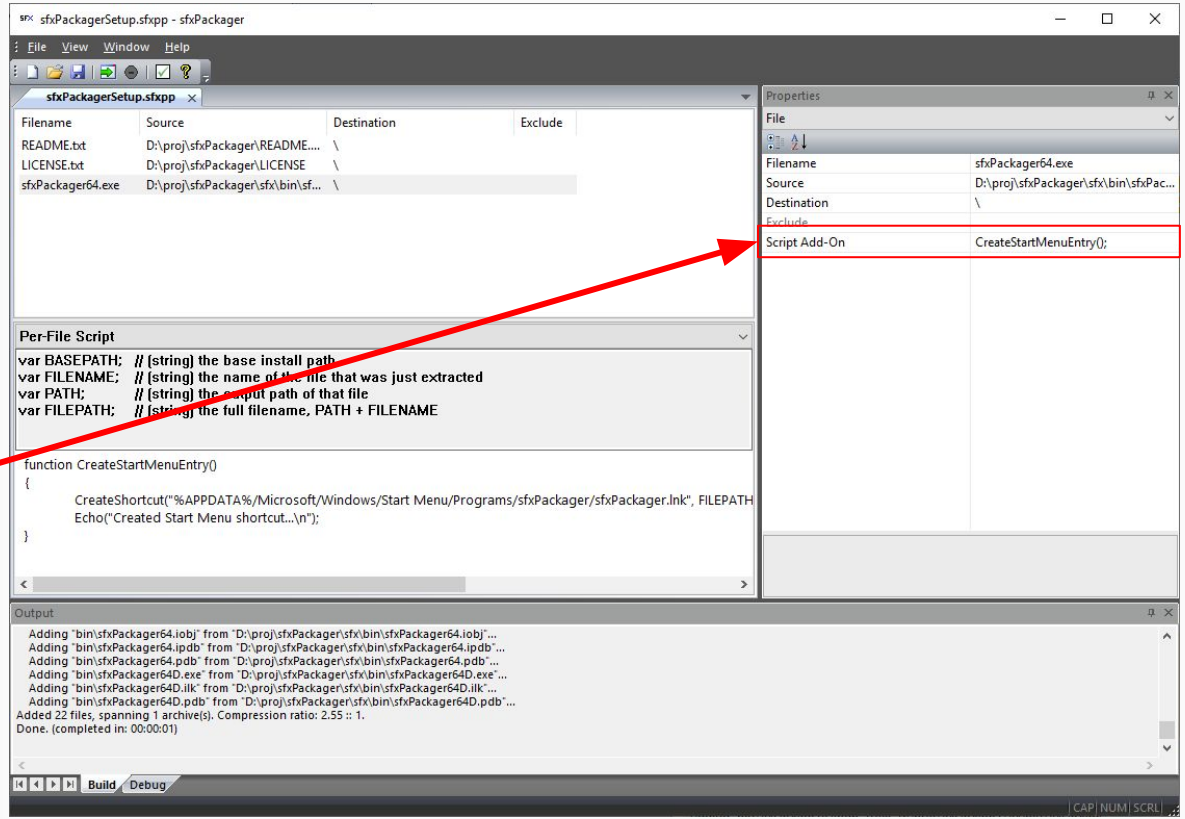
Select the type of script you wish to edit here. The corresponding code will then appear in the code editor.

Above the text editor itself is a list of system-defined, read-only global variables that are provided for that script type that you may use as you wish.



Per-File Scripts

There two Per-File scripts available to you: pre-install and post-install. The former runs before a file is installed and can allow you to Skip() that file, while the latter runs after the file has been installed. In order to run special, different code for multiple groups of files, it is recommended that you write functions in the Per-File Scripts and call them from the Snippets, which are appended to the full scripts.



Complex Configurations / User Options

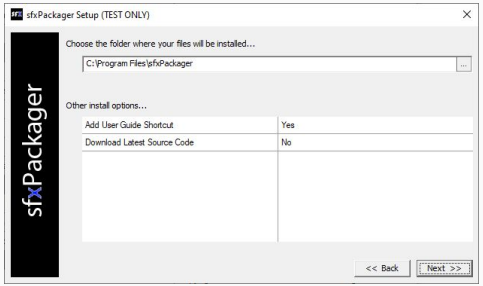
If you have the need to add more complex options to your user, it's easy: use the SetProperty function in your Initialization script, then retrieve the values using GetProperty in your other scripts. Properties you add will appear in the configuration dialog along with the install path editor.

For example, in the sfxPackager setup, one of the options is to add an internet shortcut to this User's Guide to the Start Menu. This is the Initialization code:

```
SetProperty("Add User Guide Shortcut", "bool", "yesno", 1);
```

In the Post-Install script, it is used this way:

```
if ((INSTALLOK == 1) && (GetProperty("Add User Guide Shortcut") == 1))
{
      /* add the shortcut */
}
```



Built-In JavaScript Functions

function AbortInstall()

Immediately aborts the installation and closes the program

function CompareStrings(str1, str2)

Performs a case-sensitive string comparison of the two arguments
Functions identically to strcmp(), returning which string is "greater" as an int

function CopyFile(src, dst)

Copys a file on disk to another location. Sub-strings bounded by '%' and '@' will be evaluated as environment variables or registry keys, respectively.

For example:
CopyFile("file.txt", "%USERPROFILE%\desktop"); // copies a file to the users desktop

function CreateDirectoryTree(path)

Builds a directory tree recursively, creating any levels that are missing

Built-In JavaScript Functions (cont. 2)

function CreateShortcut(file, target, args, rundir, desc, showmode, icon, iconidx)

Creates a shortcut. Useful for adding items to the Start Menu or Desktop, etc.

For example:
// FILEPATH and BASEPATH are pre-defined VARs in per-file scripts
CreateShortcut("%APPDATA%/Microsoft/Windows/Start Menu/Programs/sfxPackager/sfxPackager.lnk", FILEPATH, "", BASEPATH, "sfxPackager (64-bit) install creation utility", 1, FILEPATH, 0);

function CreateSymbolicLink(linkfile, linktarget)

Creates a symbolic link to another file or folder.

function DeleteFile(path)

Deletes a file on disk

function DeleteRegistryKey(root, key, subkey)

Deletes a registry key

Built-In JavaScript Functions (cont. 3)

function DownloadFile(url, file)

Downloads a file from the given URL to the given file. Could also be used to make general HTTP requests.

function Echo(msg)

Displays a message in the progress dialog's log display.

function FileExists(path)

Returns 1 if the file exists, 0 if it does not

function FilenameHasExtension(filename, ext)

Returns 1 if the given filename has the same extension as ext, 0 if it does not

function GetCurrentDateString()

Returns the current date as a string in YYYY/MM/DD format

Built-In JavaScript Functions (cont. 4)

function GetGlobalEnvironmentVariable(varname)

Returns the value of a program global int according to its name.

function GetGlobalInt(name)

Returns the value of a program global int according to its name. You can define these key-value pairs using the SetGlobalInt function.

function GetExeVersion(file)

Looks at the File Version Info information and returns a string in major.minor.revision.build form

function GetFileNameFromPath(filepath)

*Returns the string that represents the **file** part of a path*

function GetLicenseKey()

Returns the KEY text entered by the user in the license dialog (See ShowLicenseDlg for more information)

function GetLicenseOrg()

Returns the ORGANIZATION text entered by the user in the license dialog (See ShowLicenseDlg for more information)

Built-In JavaScript Functions (cont. 5)

function GetLicenseUser()

Returns the USER text entered by the user in the license dialog (See ShowLicenseDlg for more information)

function IsDirectory(path)

Returns 1 if the given path refers to an actual directory, 0 otherwise

function IsDirectoryEmpty(path)

Returns 1 if the given directory has no files in it, 0 otherwise

function MessageBox(title, msg)

Displays a standard Windows message box with an OK button

function MessageBoxYesNo(title, msg)

Displays a standard Windows message box with YES and NO buttons
Returns 1 is "YES" was clicked, 0 if "NO" (or close) was clicked

Built-In JavaScript Functions (cont. 6)

function RegistryKeyValueExists(root, key, name)

Returns 1 if the given registry value exists, 0 if not
root is one of {"HKEY_CURRENT_USER", "HKEY_CURRENT_CONFIG", "HKEY_LOCAL_MACHINE"}
key is the key name is the valuename

function RenameFile(filename, newname)

Renames the given file to the new name

function SetGlobalEnvironmentVariable(varname, val)

This allows you to set the environment variable varname to the given val,
for example, you could give "MY_APP_DIR" a value of BASEPATH.
This may require a reboot if currently-running processes need their
environment updated.

function SetGlobalInt(name, val)

Sets the sfx-script global integer, given by name, to the given value val

Built-In JavaScript Functions (cont. 7)

function SetProperty(name, type, aspect, value)

Sets / creates a property that will be displayed in the install options dialog. Most useful for giving your end users custom install options. Primarily used in your Initialization script.

type { "int", "float", "guid", "enum", "bool" }: determines the basic underlying type
aspect { "filename", "directory", "rgb", "rgba", "onoff", "yesno", "truefalse", "abled", "font", "date", "time", "ipaddr" }: directs the property editor as to how it should display / edit the property

function GetProperty(name)

Returns the value of the requested property. Adhering to the JS type convention, you will receive a string (string and guid types), float, or int (int, enum, and bool types).

Built-In JavaScript Functions (cont. 8)

function SetRegistryKeyValue(root, key, name, val)

Sets the registry value to a string you provide
root is one of {"HKEY_CURRENT_USER", "HKEY_CURRENT_CONFIG", "HKEY_LOCAL_MACHINE"}
key is the key name is the valuename

function ShowLicenseAcceptanceDlg()

Displays a modal dialog that allows the user to accept the given license agreement before continuing.
note: if the user cancels this dialog, the process exits immediately

function ShowLicenseDlg()

Displays a modal dialog that allows the user to enter a user name, organization, and license key
The values entered in the dialog can be obtained through the GetLicense functions*
note: if the user cancels this dialog, the process exits immediately

function SpawnProcess(cmd, params, rundir, block)

Spawns a process, given a command, list of parameters, a directory to run out of, and whether
the install process should wait for the process to complete before continuing

function TextFileOpen(filename, mode)

Opens filename as text. Specify "r" for mode to read, "w" to write
Returns a value that is passed into other Text functions as handle*

Built-In JavaScript Functions (cont. 9)

function TextFileClose(handle)

Closes a file previously opened with TextFileOpen

function TextFileReadLn(handle)

Returns a full line of text read from a file opened with TextFileOpen

function TextFileWrite(handle, text)

Writes some text to a file opened with TextFileOpen

function TextFileReachedEOF(handle)

Returns true if the end of the file has been reached

function ToLower(str)

Converts str to lower-case and returns the result as another string

function ToUpper(str)

Converts str to upper-case and returns the result as another string